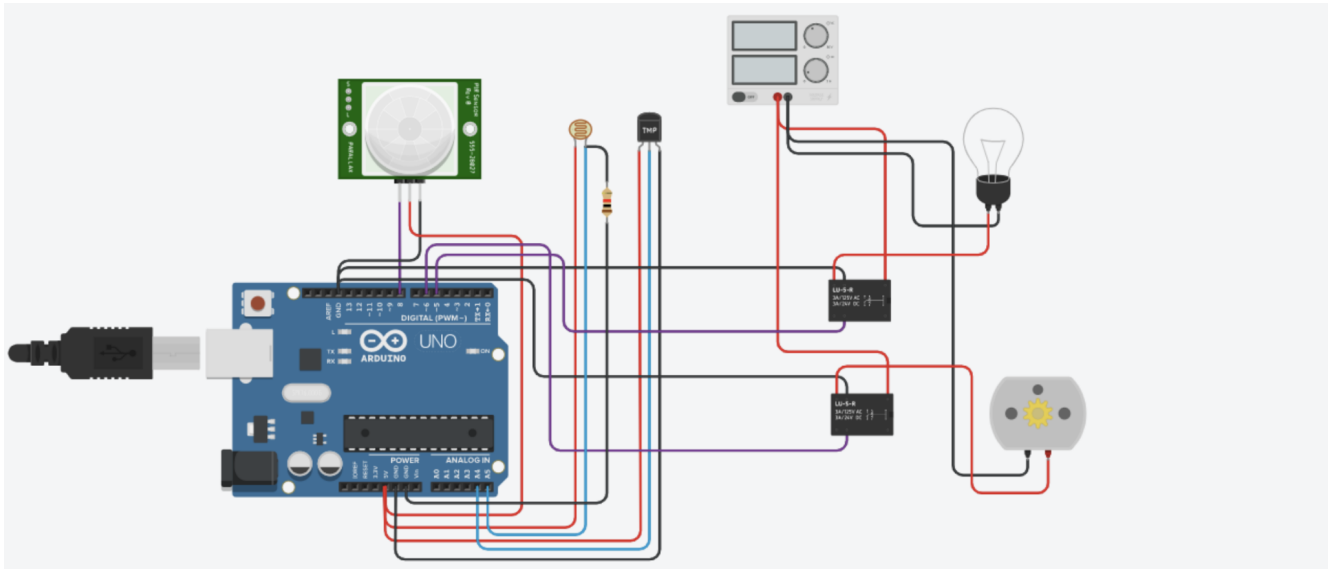


# Smart IoT Automation System

## Smart Temperature Control / Smart AC & Server Room

### Project objective

This project upgrades a sensor circuit into an automatic smart system. A TMP36 temperature sensor monitors temperature and automatically controls a fan through a relay. A PIR sensor and LDR can also automate the room light through a second relay. The design demonstrates sensing, decision making, automatic control, and monitoring.



**Reference circuit supplied:** Arduino UNO with PIR, LDR, TMP temperature sensor, two relay modules, lamp and DC motor/fan.

### Suggested pin mapping for the shown circuit

| Component          | Arduino pin | Purpose                    |
|--------------------|-------------|----------------------------|
| TMP36 output       | A1          | Temperature measurement    |
| LDR divider output | A0          | Light level measurement    |
| PIR output         | D8          | Motion detection           |
| Fan relay IN       | D7          | Temperature-controlled fan |
| Light relay IN     | D6          | Automatic light            |

**Important:** The uploaded image does not make every Arduino pin label unambiguous. The code below uses the pin mapping above. If your Tinkercad wiring uses different pins, change only the constants at the top of the code.

# Arduino Code

```
/*
  SMART IoT AUTOMATION SYSTEM
  Smart AC / Server Room

  Sensors:
    TMP36 -> A1
    LDR    -> A0
    PIR    -> D8

  Outputs:
    Fan relay    -> D7
    Light relay  -> D6

  Logic:
    Temperature >= 30 C -> Fan ON
    Temperature <= 28 C -> Fan OFF
    Dark + motion detected -> Light ON
    Otherwise -> Light OFF

  NOTE:
    Many relay modules are ACTIVE-LOW.
    Set RELAY_ON / RELAY_OFF below according to your module.
*/

const int LDR_PIN = A0;
const int TMP36_PIN = A1;
const int PIR_PIN = 8;

const int FAN_RELAY_PIN = 7;
const int LIGHT_RELAY_PIN = 6;

// Change these if your relay module is active-low/active-high.
const int RELAY_ON = LOW;
const int RELAY_OFF = HIGH;

// Temperature thresholds (hysteresis prevents rapid relay switching)
const float FAN_ON_TEMP = 30.0;
const float FAN_OFF_TEMP = 28.0;

// LDR threshold: lower reading usually means darker.
// Adjust after checking Serial Monitor.
const int DARK_THRESHOLD = 450;

bool fanState = false;
bool lightState = false;

float readTMP36C() {
  int raw = analogRead(TMP36_PIN);
  float voltage = raw * (5.0 / 1023.0);
  return (voltage - 0.5) * 100.0;
}

void setFan(bool on) {
  fanState = on;
  digitalWrite(FAN_RELAY_PIN, on ? RELAY_ON : RELAY_OFF);
}

void setLight(bool on) {
  lightState = on;
  digitalWrite(LIGHT_RELAY_PIN, on ? RELAY_ON : RELAY_OFF);
}

void setup() {
  pinMode(PIR_PIN, INPUT);
  pinMode(FAN_RELAY_PIN, OUTPUT);
  pinMode(LIGHT_RELAY_PIN, OUTPUT);

  setFan(false);
  setLight(false);

  Serial.begin(9600);

  Serial.println("Smart IoT Automation System");
  Serial.println("Temperature + Light + Motion Control");
}

void loop() {
  float temperatureC = readTMP36C();
  int ldrValue = analogRead(LDR_PIN);
  bool motion = digitalRead(PIR_PIN) == HIGH;

  // ----- SMART TEMPERATURE CONTROL -----
  if (!fanState && temperatureC >= FAN_ON_TEMP) {
    setFan(true);
  }
}
```

```

}
else if (fanState && temperatureC <= FAN_OFF_TEMP) {
    setFan(false);
}

// ----- SMART LIGHT CONTROL -----
bool dark = ldrValue < DARK_THRESHOLD;

if (dark && motion) {
    setLight(true);
} else {
    setLight(false);
}

// ----- MONITORING / LOGGING -----
Serial.print("Temperature: ");
Serial.print(temperatureC, 1);
Serial.print(" C | LDR: ");
Serial.print(ldrValue);
Serial.print(" | Motion: ");
Serial.print(motion ? "YES" : "NO");
Serial.print(" | Fan: ");
Serial.print(fanState ? "ON" : "OFF");
Serial.print(" | Light: ");
Serial.println(lightState ? "ON" : "OFF");

delay(1000);
}

```

# How the Automation Works

- **1. Temperature monitoring:** TMP36 sends an analog voltage to A1.
- **2. Temperature decision:** At 30°C or higher, the fan relay turns ON. The fan stays ON until temperature falls to 28°C. This 2°C gap is hysteresis and reduces relay chattering.
- **3. Light automation:** The LDR detects darkness and the PIR detects motion. The lamp turns ON only when it is dark and motion is detected.
- **4. Monitoring:** The Serial Monitor logs temperature, LDR value, motion, fan state, and light state every second.
- **5. Real-world use:** The fan section represents automatic cooling in a server room, while the light section represents smart room lighting.

## Advantages

- Automatic temperature-based cooling reduces the need for manual control.
- Helps protect server/electronic equipment from excessive heat.
- PIR + LDR combination avoids turning the room light on unnecessarily during daylight or when no one is present.
- Relay isolation allows the Arduino control signal to operate a separate load circuit.
- Hysteresis reduces frequent ON/OFF switching of the fan relay.
- Serial logging provides basic real-time monitoring and debugging.
- Low-cost components make it suitable for a student mini-project and Tinkercad simulation.
- The design can be extended to Wi-Fi/IoT monitoring using an ESP32.

## Disadvantages / Limitations

- The basic Arduino version is not connected to the Internet, so it is not true cloud IoT until Wi-Fi is added.
- TMP36 temperature accuracy is limited compared with calibrated industrial sensors.
- LDR readings depend on the surrounding light and require threshold calibration.
- PIR sensors may have a short warm-up period and can occasionally trigger false detections.
- Mechanical relays have a limited switching life and are slower than solid-state switching.
- A DC motor/fan should not be powered directly from an Arduino GPIO pin; use a transistor/MOSFET or relay and a suitable power supply.
- For real AC mains appliances, proper isolation, fusing, enclosure, earthing, and qualified electrical installation are required.

## Tinkercad Testing

- Set the TMP36 temperature below 28°C → Fan OFF.
- Increase temperature to 30°C or above → Fan relay ON.
- Reduce temperature to 28°C or below → Fan relay OFF.
- Make the LDR dark and activate the PIR → Light relay ON.
- Remove motion or make the area bright → Light relay OFF.
- Open Serial Monitor at 9600 baud to observe the logged values.

**Safety note:** The uploaded diagram shows a lamp/load connected through relays. For Tinkercad, use the simulator's low-voltage components. Do not connect an Arduino directly to household mains. For a real AC server-room system, use an appropriately rated relay/contactors and professional electrical protection.